You will find the class definitions and instances for *Show* and *Read* in *libraries/base/GHC/Show.lhs* and *libraries/base/GHC/Read.lhs* respectively. The *.lhs* file is a literate Haskell source file in which you can combine both text and code. You can also find the definition for a class, a function or type inside GHCi using ':i'. For example:

```
ghci> :i Eq
class Eq a where
  (==) :: a -> a -> Bool
  (/=) :: a -> a -> Bool
    -- Defined in `GHC.Classes'
instance Eq Integer -- Defined in `integer-gmp:GHC.Integer.Type'
instance Eq Ordering -- Defined in `GHC.Classes'
instance Eq Int -- Defined in `GHC.Classes'
instance Eq Float -- Defined in `GHC.Classes'
instance Eq Double -- Defined in `GHC.Classes'
instance Eq Char -- Defined in `GHC.Classes'
instance Eq Bool -- Defined in `GHC.Classes'
...

ghci> :i read
read :: Read a => String -> a   -- Defined in `Text.Read'

ghci> :i Int
data Int = GHC.Types.I# GHC.Prim.Int#   -- Defined in `GHC.
Types'
instance Bounded Int -- Defined in `GHC.Enum'
instance Enum Int -- Defined in `GHC.Enum'
instance Eq Int -- Defined in `GHC.Classes'
instance Integral Int -- Defined in `GHC.Real'
instance Num Int -- Defined in `GHC.Num'
instance Ord Int -- Defined in `GHC.Classes'
instance Read Int -- Defined in `GHC.Read'
instance Real Int -- Defined in `GHC.Real'
instance Show Int -- Defined in `GHC.Show'
```

Let's suppose you input the following in a GHCi prompt:

```
ghci> read "3"

<interactive>:5:1:
    No instance for (Read a0) arising from a use of `read'
    The type variable `a0' is ambiguous
    Possible fix: add a type signature that fixes these type
variable(s)
    Note: there are several potential instances:
      instance Read () -- Defined in `GHC.Read'
      instance (Read a, Read b) => Read (a, b) -- Defined in
`GHC.Read'
      instance (Read a, Read b, Read c) => Read (a, b, c)
        -- Defined in `GHC.Read'
      ...plus 25 others
    In the expression: read "3"
    In an equation for `it': it = read "3"
```

The interpreter does not know what type to convert '3' to, and hence you will need to explicitly specify the type:

```
ghci> read "3" :: Int
3
ghci> read "3" :: Float
3.0
```

A type synonym is an alias that you can use for a type. 'String' in the Haskell platform is an array of characters defined using the *type* keyword:

```
type String = [Char]
```

You can also create a new user data type using the *data* keyword. Consider a *Weekday* data type that has the list of the days in a week:

```
data Weekday = Monday
             | Tuesday
             | Wednesday
             | Thursday
             | Friday
             | Saturday
             | Sunday
```

The *data* keyword is followed by the name of the data type, starting with a capital letter. After the 'equal to' ('=') sign, the various value constructors are listed. The different constructors are separated by a pipe ('|') symbol.

If you load the above data type in GHCi, you can test the value constructors:

```
ghci> :t Monday
Monday :: Weekday
```

Each value constructor can have many type values. The user defined data type can also derive from type classes. Since the primitive data types already derive from the basic type classes, the user defined data types can also be derived. Otherwise, you will need to write instance definitions for the same. The following is an example for a user data type 'Date' that derives from the 'Show' type class for displaying the date:

```
data Date = Int String Int deriving (Show)
```

Loading the above in GHCi, you get:

```
ghci> Date 3 "September" 2014
Date 3 "September" 2014
```

The above code will work even if we swap the year and